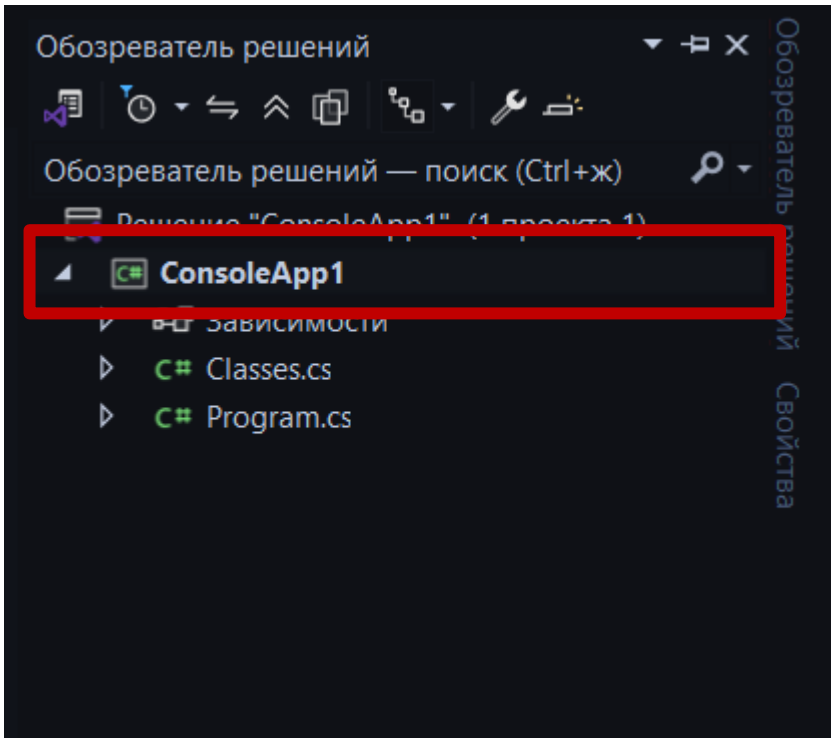
Создание окна каталога товаров +

Фильтрация, поиск и сортировка

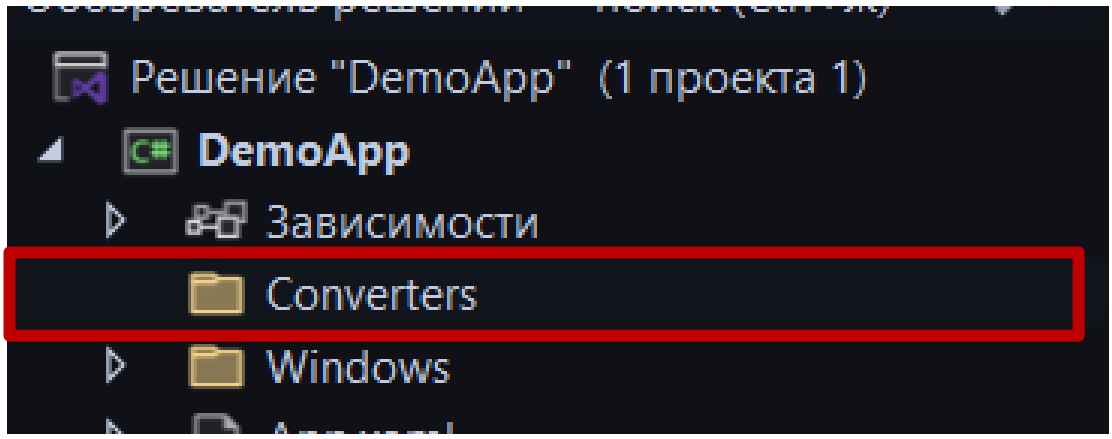
у всех методов, которые используются только внутри класса (окна) модификатор доступа *private*, также методы не возвращают значений, то есть указывается *void*.

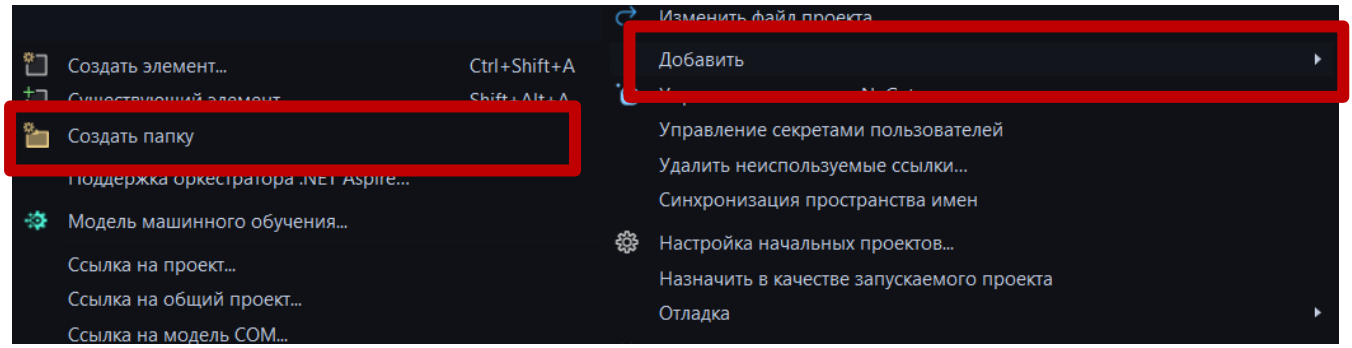
1. Создайте папку Converters.

1.1 Откройте обозреватель решений, кликните ПКМ (правой кнопкой мыши по проекту)

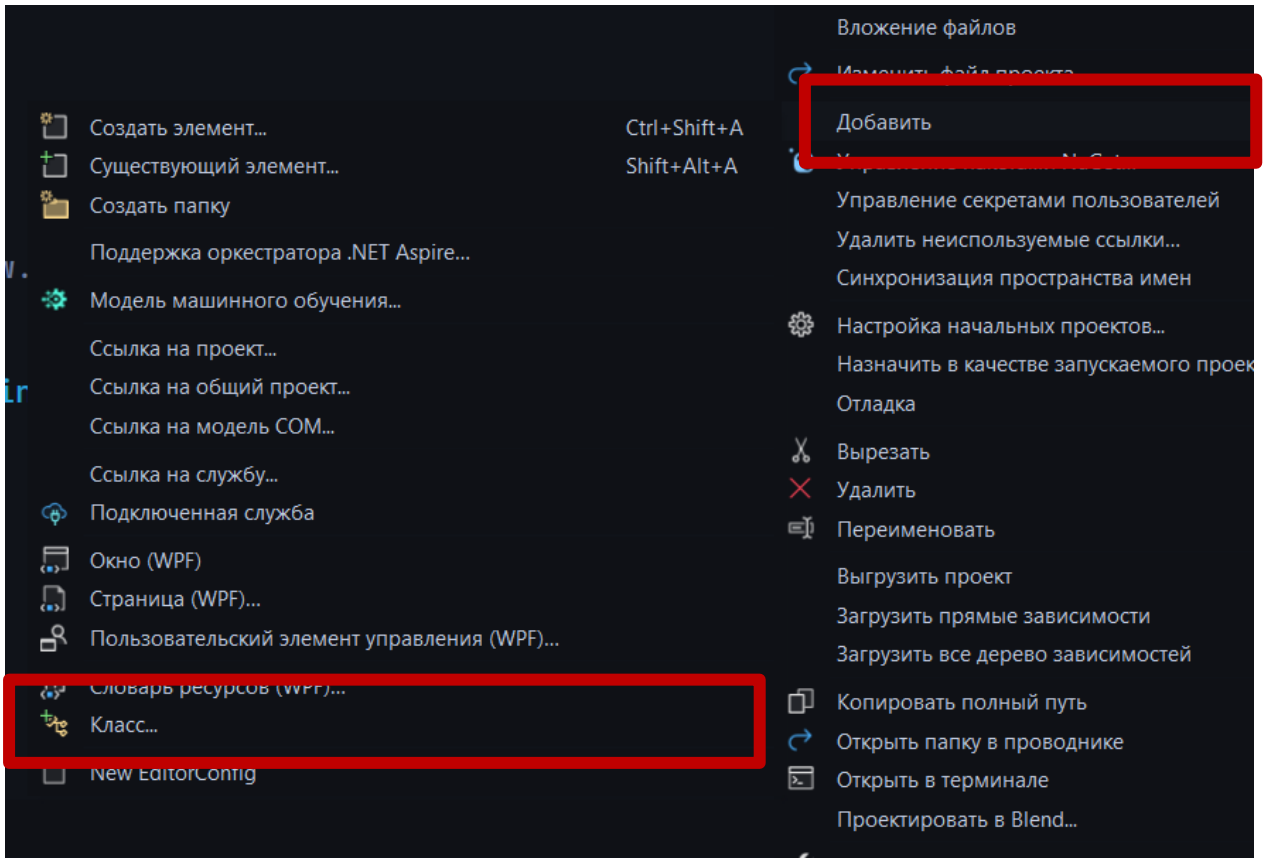


Создайте папку, для этого выберите «Добавить» → «Папку». После чего переименуйте, нажав по папке ПКМ → «Переименовать» или нажав F2, предварительно кликнув на папку.





2. Создайте 4 класса в папке, для этого кликните по ней ПКМ → «Добавить» → «Класс»



2.1 В появившемся окне присвойте название классу **(НЕ ОСТАВЛЯЙТЕ CLASS1 И ТД, ДАВАЙТЕ ПРАВИЛЬНЫЕ НАЗВАНИЯ ДЛЯ ФАЙЛОВ И КЛАССОВ)**

3. Поменяйте модификатор доступа на public и наследуйте IValueConverter

```
class ImagePathConverter
{
}
```

```
Ссылок: 0
public class ImagePathConverter : IValueConverter
{
}
```

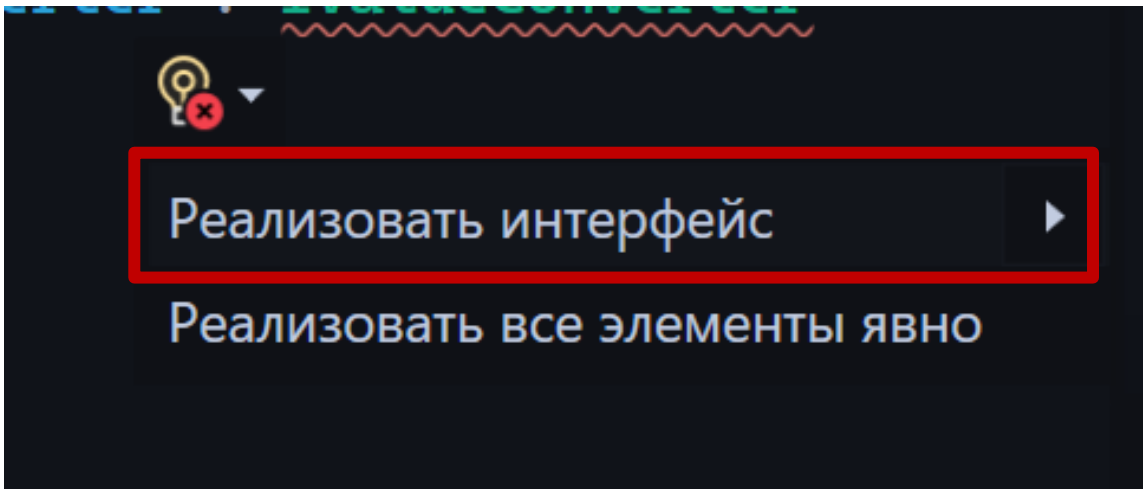
Чтобы не прописывать необходимые методы вручную, наведите мышку на название интерфейса (IValueConverter), и нажмите на лампочку и выберите пункт «Реализовать интерфейс»

```
ter : IValueConverter
interface System.Windows.Data.IValueConverter
Provides a way to apply custom logic to a binding.

CS0535: "ImagePathConverter" не реализует член интерф
"IValueConverter.Convert(object, Type, object, CultureInfo)"

CS0535: "ImagePathConverter" не реализует член интерф
"IValueConverter.ConvertBack(object, Type, object, Culture

Показать возможные решения (Alt+ВВОДилиCtrl+.)
```



У вас появятся 2 метода

```
Ссылок: 0
public class ImagePathConverter : IValueConverter
{
    Ссылок: 0
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }

    Ссылок: 0
    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}
```

Метод ConvertBack остается **НЕИЗМЕННЫМ**.

В методе Convert сотрите исключение и пропишите необходимый код **(файлы: конвертеры.pdf и конвертеры (карточки).pdf)**

Необходимо сделать 4 Конвертера:

- ImagePathConverter — конвертер изображения
- SaleToBackgroundConverter — конвертер фона скидки
- CrossedPriceConverter — конвертер перечеркнутой цены
- FinalPriceConverter — конвертер итоговой цены

4. В пространствах окна (в любое место)

```
<Window
    x:Class="DemoApp.Windows.CatalogWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:local="clr-namespace:DemoApp.Windows"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    Title="Каталог"
    Width="860"
    Height="770"
    Icon="/Images/Icon.ico"
    ResizeMode="NoResize"
    WindowStartupLocation="CenterScreen"
    mc:Ignorable="d">
```

Укажите путь к конвертерам, например:

```
xmlns:conv="clr-namespace:ShoeShop.Converters"
```

5. В ресурсах окна (прописываются после тега Window и перед тегом Grid)

```
    Title="MainWindow" Height="450" Width="800">

<Window.Resources>

</Window.Resources>

<Grid>

</Grid>
```

(<Window.Resources> **ЭЛЕМЕНТЫ** </Window.Resources>) укажите <conv:ImagePathConverter x:Key="ImagePathConverter" />, а также остальные конвертеры. !!!НАЗВАНИЯ ПРИМЕРНЫЕ АДАПТИРУЙТЕ ПОД СЕБЯ!!!

После слова conv: указывается название конвертера, которое прописано у класса, а после x:Key название, по которому можно будет обращаться в коде.

Пропишите стили для Border’а и ListBoxItem

Для Border’а установите ключ, а также свойства:

- Цвет границ (BorderBrush) цвет должен быть черный
- Толщина границ (BorderThickness) значение 2
- Внешние отступы (Margin)
- Внутренние отступы (Padding)

Для ListBoxItem:

Установите фон (Background) прозрачный (Transparent)

Также внутри стиля пропишите Триггер

```
<Style.Triggers>
     <DataTrigger Binding="{Binding Count}" Value="0">
         <Setter Property="Background" Value="LightBlue" />
    </DataTrigger>
</Style.Triggers>
```

Здесь мы ссылаемся на поле количества товара, если оно будет равно нулю, то фон будет окрашен в светло-голубой.

РАЗМЕТКА ОКНА

Для удобства реализации рекомендуется прописать стили для кнопок, шрифта и рамок (Border).

1. Измените название окна (Title) на «Каталог товаров», установите его размеры (Width и Height) (можно установить Width 860 и Height 770), иконку (Icon), и разместите начальную позицию в центре (WindowStartUpLocation), а также ограничьте изменение размеров окна (ResizeMode).

2. Сетку разделите на 4 строки (RowDefinitions). Для каждой, кроме третьей установите высоту «auto», для третьей строки пропорциональную высоту («*»).

3. Пропишите Border, разместите его в первой строке (Grid.Row = «0»), окрасьте в цвет из руководства по стилю.

3.1 В Border’е разместите сетку с двумя столбцами (ColumnDefinitions).

3.2 В первом столбце разместите картинку с логотипом комапании, выровняйте по левому краю (HorizontalAlignment) и установите отступы (Margin), слева и справа 15, снизу и сверху 0.

3.3 Во втором столбце разместите StackPanel и выровняйте по правому краю (HorizontalAlignment).

3.4 В него поместите TextBlock для отображения ФИО пользователя и кнопку для выхода, не забудьте указать имена (x:Name) для обращения в коде. Имена прописывайте содержательные (например, UserNameTB, ExitBtn и тд).

3.5 Для кнопки можно указать курсор «Hand», а также необходимо создать обработчик события (Click).

**** НЕ НУЖНО ПРОПИСЫВАТЬ НАЗВАНИЕ МЕТОДА, VS ПРЕДЛОЖИТ СОЗДАТЬ НОВЫЙ ОБРАБОТЧИК СОБЫТИЙ. НАЖМИТЕ TAB. VS АВТОМАТИЧЕСКИ ВПИШЕТ ПРИВЯЗКУ И СОЗДАСТ МЕТОД В C# - ФАЙЛЕ.**

```
<StackPanel>
    <Button Click=""/>
</StackPanel>
```

<Новый обработчик событий>

4. Вне Boder’a создайте сетку и разместите во второй строке (Grid.Row = «1»), присвойте ей имя (x:Name). Сетку разделите на 3 столбца (ColumnDefinitions). Сетка будет служить для элементов сортировки, фильтрации и поиска.

4.1 В первом столбце пропишите StackPanel (Grid.Column = «1»), в нем разместите TextBlock и присвойте текст: «Поиск». Следом пропишите TextBox, присвойте ему имя (x:Name).

4.2 Во втором столбце разместите ComboBox, присвойте ему имя (x:Name). Это будет список для поставщиков. Также пропишите свойства:

DisplayMemberPath = «Поле с названиями поставщиков»

SelectedValuePath="Айди поставщика"

Названия полей берутся из классов-сущностей. (папка Entities)

4.3 В третьем столбце (Grid.Column = «2») разместите ComboBox (тег закрывается
< ComboBox> </ ComboBox >), присвойте ему имя (x:Name). Это будет список для фильтрации по количеству товара на складе. Также пропишите ComboBoxItem (для каждого элемента отдельно. Для того, чтобы присвоить текст, пропишите Content):

Без сортировки (пропишите свойство IsSelected = True)

По возрастанию

По убыванию

Или подобным образом

4.4 После того, как пропишите разметку скройте сетку: свойство Visibility = «Collapsed».

```
<Grid Visibility="Collapsed">
```

5. Вне сетки сортировки пропишите Border, присвойте стиль и разместите его в третьей строке (Grid.Row = «2»). Нам необходимо выводить товары по следующему шаблону:

5.1 Внутри разместите ListBox, присвойте ему имя и скройте вертикальную прокрутку (ScrollViewer.VerticalScrollBarVisibility = «Hiden» (Не обязательно, но выглядит лучше)), а также уберите границы (BorderThikcness = «0»)

5.2 Создайте привязку данных:

```
<ListBox.ItemTemplate>
<DataTemplate>
<!-- СОДЕРЖИМОЕ-->
</DataTemplate>
</ListBox.ItemTemplate>
```

5.3 Внутри <DataTemplate> создайте сетку и разделите ее на три столбца (ColumnDefinitions), ширину столбцов подбирайте в процессе верстки. Если используйте размеры окна из примера, то для первого столбца можно установить ширину 200, для второго 500, а для третьего 100.

5.4 Создайте Border и поместите его в первом столбце (Grid.Column = «0»), также примените стиль.

5.5 Внутри Border'а разместите изображение, в качестве ресураса привязка данных, сюда же установите конвертер для изображения:
{Binding Название_поля_с_изображением,
Converter={StaticResource Название_конвертера}}

5.6 Создайте второй Border и поместите его во втором столбце (Grid.Column = «1»), в нем расположите StackPanel.

5.7 В StackPanel выведите всю необходимую информацию через привязку данных, которая указана в шаблоне. Для того, чтобы вместе с привязкой выводился дополнительно текст, используйте StringFormat, например:

```
<TextBlock Text="{Binding Sup.SupName, StringFormat='Поставщик: {0}'}" />
```

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СВОИ ТАБЛИЦЫ

Для вывода «Категория товара | Наименование товара» используйте следующую конструкцию:

```
<TextBlock FontWeight="Bold">
<Run Text="{Binding ТаблицаКатегории.ПолеНазванияКатегории}" />
| <Run Text="{Binding ТаблицаНазвания.ПолеНазванияТовара }" />
</TextBlock>
```

Если данные находятся в отдельных таблицах, то необходимо сначала указать таблицу, и через точку указать поле с названием. Посмотреть названия таблиц можно в классе таблицы товаров.

```
Ссылок: 3
public virtual Category Cat { get; set; } = null!;

Ссылок: 3
public virtual Manufacrurer Manuf { get; set; } = null!;

Ссылок: 3
public virtual Prodname ProdNameNavigation { get; set; } = null!;

Ссылок: 1
public virtual ICollection<Prodorder> Prodorders { get; set; } = new List<Prodorder>();

Ссылок: 3
public virtual Supplier Sup { get; set; } = null!;
```

Класс товаров

Для отображения цены пропишите горизонтальный StackPanel, пропишите 2 TextBlock’a и.

В первом TextBlock’e сделайте привязку данных и укажите конвертер итоговой цены.

Text="{Binding Converter={StaticResource FinalPriceConverter}}"

Второй TextBlock окрасьте в красный (Foreground), сделайте привязку цены. Также укажите отдельно свойство TextDecorations и сделайте привязку данных, в качестве значения укажите поле со скидкой и укажите конвертер с перечеркнутой ценой.

Text="{Binding Price, StringFormat='{}{0:N0} руб'}"

TextDecorations="{Binding Sale,

Converter={StaticResource CrossedPriceConverter}}"

5.8 В третьем столбце разместите еще один Border, в нем разместите скидку, примените конвертер. Конвертер применяется для фона Border’a

Background="{Binding Sale,

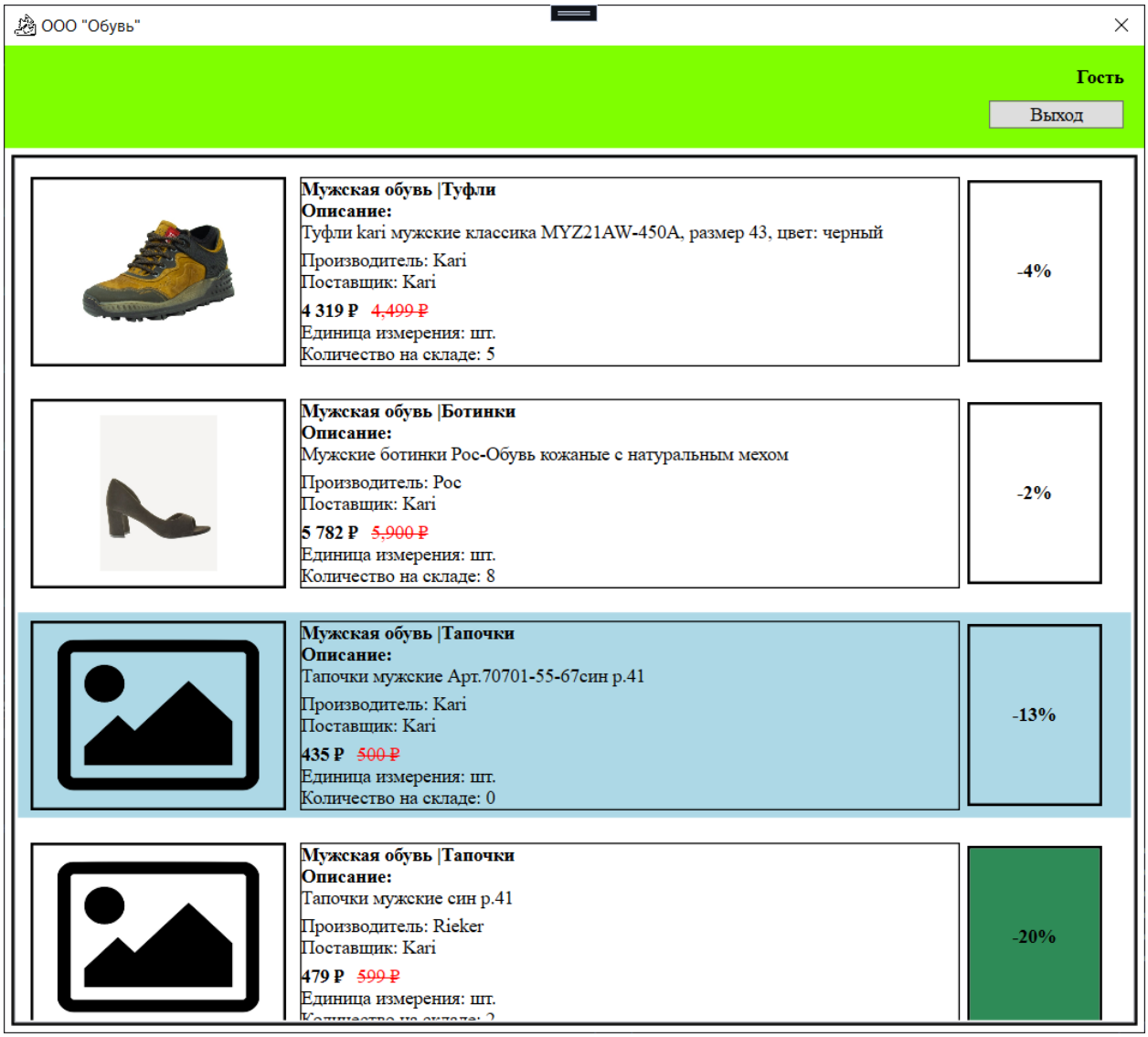
Converter={StaticResource SaleToBackgroundConverter}}"

6. В четвертой строке (Grid.Row = «3») разместите сетку и присвойте ей имя (x:Name). Это будет админ-панель. Разделите сетку на три столбца (ColumnDefinitions). Разместите 3 кнопки: для добавления, редактирования и удаления товара.

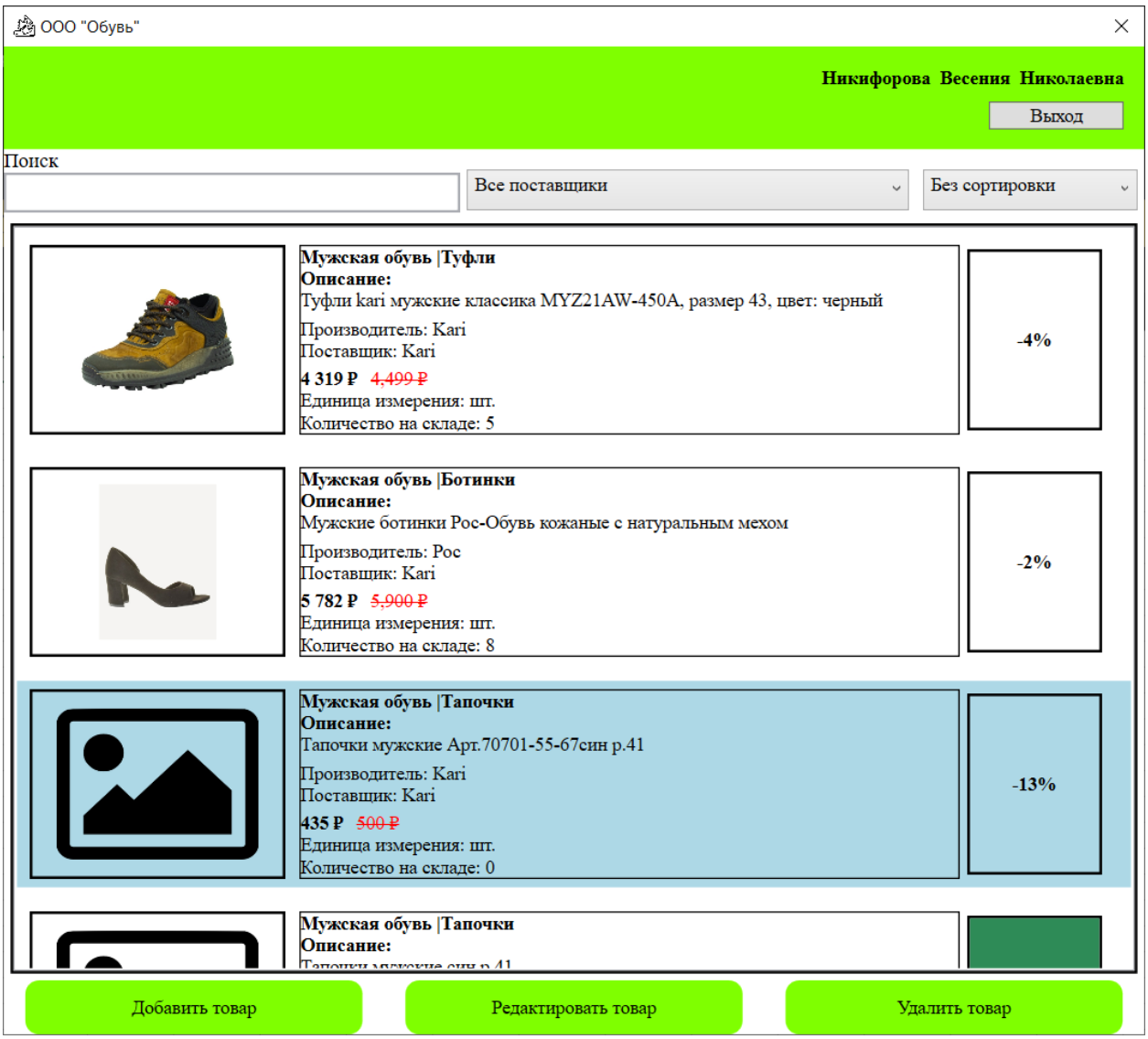
Кнопкам пропишите имя (x:Name), текст (Content), примените стиль, и припишите обработчик событий (Click)

После того, как пропишите разметку скройте сетку: свойство Visibility = «Collapsed».

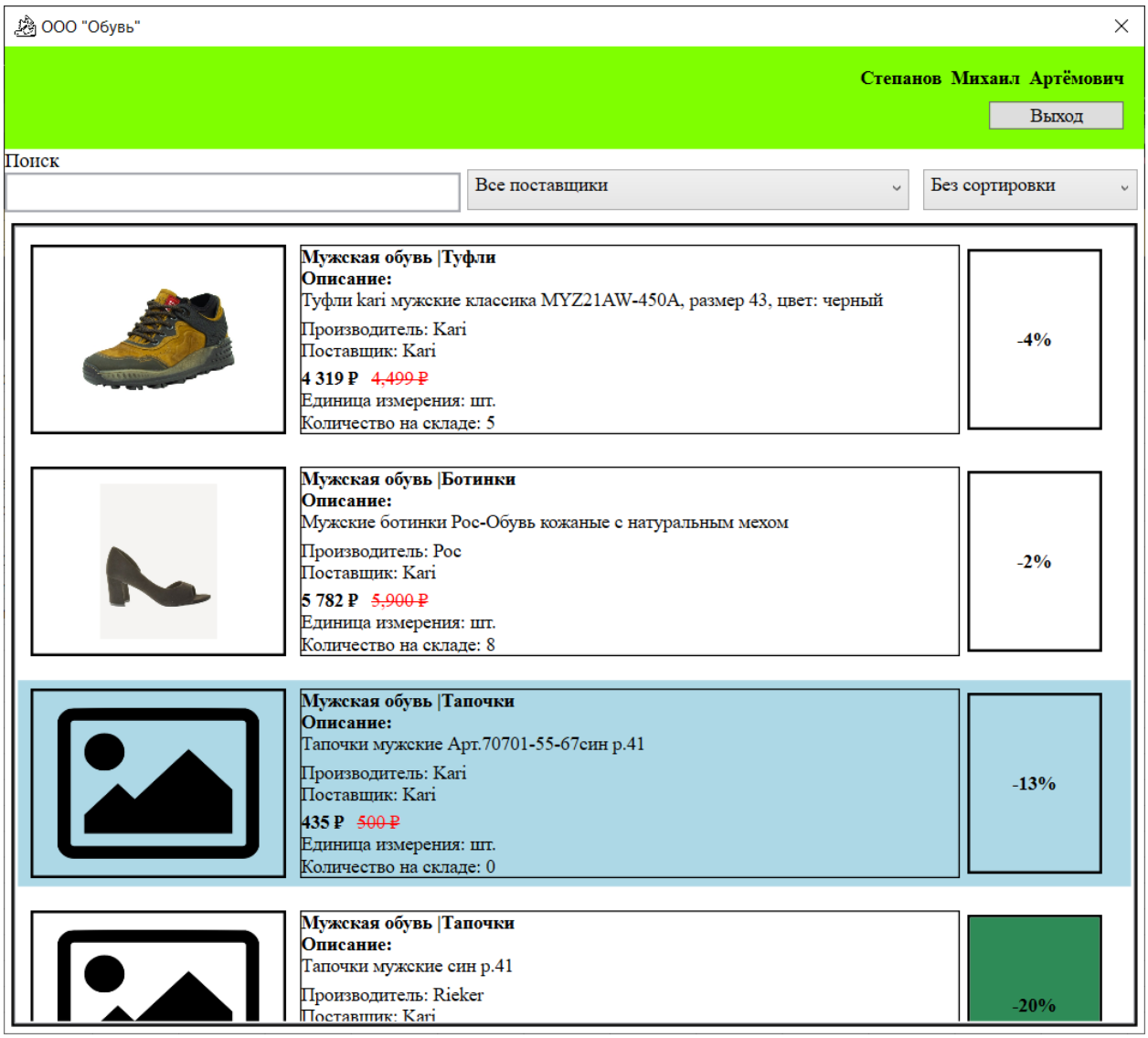
!!!ПРИМЕР!!!



Гость и авторизованный клиент



Администратор


ООО "Обувь"
Степанов Михаил Артёмович
Выход
Поиск
Все поставщики
Без сортировки
Мужская обувь |Туфли
Описание:
Туфли kari мужские классика MYZ21AW-450A, размер 43, цвет: черный
Производитель: Kari
Поставщик: Kari
4 319 ₽ 4,499 ₽
Единица измерения: шт.
Количество на складе: 5
-4%
Мужская обувь |Ботинки
Описание:
Мужские ботинки Рос-Обувь кожаные с натуральным мехом
Производитель: Рос
Поставщик: Kari
5 782 ₽ 5,900 ₽
Единица измерения: шт.
Количество на складе: 8
-2%
Мужская обувь |Тапочки
Описание:
Тапочки мужские Арт.70701-55-67син р.41
Производитель: Kari
Поставщик: Kari
435 ₽ 500 ₽
Единица измерения: шт.
Количество на складе: 0
-13%
Мужская обувь |Тапочки
Описание:
Тапочки мужские син р.41
Производитель: Rieker
-20%

Менеджер

ЛОГИКА ОКНА

1. Откройте C# - файл (расширение *.xaml.cs) окна каталога товаров.

2. В конструкторе окна

```
public CatalogWindow()
{
   InitializeComponent();
}
```

Присвойте TextBlock’у с отображением ФИО имя текущего пользователя. Не забудьте для TextBlock’а использовать свойство .Text.

```
UserNameTB.Text = CurrentUser.FullName
```

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

3. Пропишите метод для отображения сеток, которым мы прописали свойство Visibility=Collapsed, по ролям (используйте тернарный оператор)

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

```
AdminPanel.Visibility =
            CurrentUser.Role == "Администратор"
             ? Visibility.Visible
             : Visibility.Collapsed;

          SortGrid.Visibility =
             CurrentUser.Role == "Администратор" ||
             CurrentUser.Role == "Менеджер"
             ? Visibility.Visible
             : Visibility.Collapsed;
```

4. Пропишите метод для загрузки поставщиков, например, LoadSupliers.

4.1 Для избегания исключительных ситуаций, используйте конструкцию try-catch.

4.2 Используйте контекст подключения к БД.

4.3 Создайте переменную и поместите туда список поставщиков.

4.4 Добавьте нового поставщика в список:

```
suppliers.Insert(0, new Supplier
{
  IdSup = 0,
  SupName = "Все поставщики"
});
```

4.5 Привяжите получившийся список к ComboBox’у поставщиков (с помощью ItemsSource).

4.6 Для ComboBox’а установите свойство

```
ComboBox.SelectedIndex = 0
```

*** Вместо ComboBox укажите название из разметки**

4.7 В блоке catch выведите сообщение об ошибке.

5. Пропишите метод в конструкторе окна.

6. Создайте метод для отображения товаров

6.1 Пропишите проверку условия на нулевое значение (null) списка (название из разметки), ComboBox’а поставщиков, ComboBox’а сортировки поколичеству, строки поиска (используйте логическое или (||)).

if (prodList == null || SupplierCB == null || SortCB == null || SearchTB == null)

Если условие выполняется, то пропишите return.

6.2 Пропишите использование контекста БД. (using var db = new КонтекстБД())

6.3 Создайте переменную (используйте неявную типизацию var) и присвойте ей: таблицу товаров, подгружая (Include) связанные таблицы:

– Категория
– Производитель
– Поставщик
– Название товара (если вынесли в отдельную таблицу)

Названия также можно посмотреть в классе товаров.

В конце пропишите .AsQueryable() для динамических запросов.

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

```
var query = db.Products
   .Include(p => p.Cat)
   .Include(p => p.Manuf)
   .Include(p => p.Sup)
   .Include(p => p.ProdNameNavigation)
   .AsQueryable();
```

6.4 Пропишите условие, если строка поиска не нулевая (!string.IsNullOrEmpry(СтрокаПоиска.Text)), то создайте строковую переменную и присвойте ей текст из строки поиска и приведите к нижнему регистру (СтрокаПоиска.Text.ToLower())

6.5 К переменной запроса, в которую происходила выборка всех таблиц, связанных с таблицей товаров присвойте:

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

```
query = query.Where(p =>
  p.Descrip.ToLower().Contains(text) ||
  p.Cat.CatName.ToLower().Contains(text) ||
  p.Manuf.ManufName.ToLower().Contains(text) ||
  p.Sup.SupName.ToLower().Contains(text) ||
  p.ProdNameNavigation.ProdName1 .ToLower().Contains(text));
```

Здесь происходит проверка на содержащиеся символы в каждом текстовом поле.

6.6 Пропишите условие на то, что выбранный элемент из ComboBox’а поставщиков является числом (ComboBox.SelectedValue is int supId && supId > 0)

*** Вместо ComboBox укажите название из разметки**

Если условие выполняется, то из запроса (переменная query) выберите те Id поставщиков, которые равны нашей переменной supId.

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

```
if (SupplierCB.SelectedValue is int supId && supId > 0)
{
   query = query.Where(p => p.SupId == supId);
}
```

6.7 Пропишите 2 отдельных условия.

6.8 В первом условии проверьте равен ли индекс ComboBox’а сортировки одному (ComboBoxСортировки.SelectedIndex == 1), если условие выполняется, то отсортируйте список (переменная query) по возрастанию (OrderBy). Сортировка осуществляется по полю количества на складе.

Так же проверьте на значение «2», в этом случае сортируйте по убыванию (OrderByDescending)

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

```
if (SortCB.SelectedIndex == 1)
   query = query.OrderBy(p => p.Count);

if (SortCB.SelectedIndex == 2)
  query = query.OrderByDescending(p => p.Count);
```

6.9 Присвойте списку в разметке (через .ItemSource) запрос.ToList().

6.10 Не забудьте обработать исключения через try-catch

7. Пропишите метод для выхода из каталога на окно авторизации (принцип такой же, как и при переходе из окна авторизации в окно каталога)

8. Вызовите все методы (кроме кнопки выхода) в конструкторе окна.

9. В разметке для поисковой строки, ComboBox’ов поставщиков и сортировки пропишите обработчик событий SelectionChanged, в C#-файле вызовите метод загрузки товаров.

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

```
private void SearchTB_TextChanged(object sender, TextChangedEventArgs e)
{
   LoadProdData();
}
```